SWC3

Software-Entwicklung mit C und C++

Erik Pitzer

WS 2026/27

FH OÖ - Campus Hagenberg - Software Engineering



1. SWC3 Überblick

»Talk is cheap.
Show me the code.«

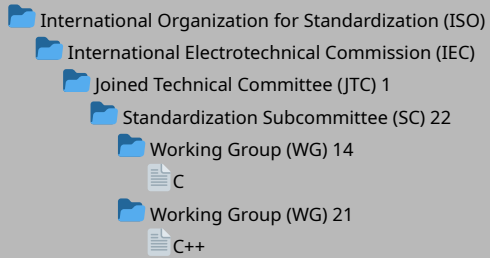
- Linus Torvalds,
Linux Kernel Mailing List, 8/2000

Klassische Programmiersprachen — Timeline

- ab 1940 Maschinenspr. und Assembler
- 1957 FORTRAN
- 1958 LISP & IAL (ALGOL 58)
- 1959 COBOL
- 1962 Simula → C++
- 1967 BCPL → B → C
- 1969 B (Ken Thomson & Dennis Ritchie)
- 1972 C (Dennis Ritchie) & Smalltalk → Obj-C
- 1978 K&R C (Kernighan & Ritchie)
- 1984 Objective-C (Brad Cox & Tom Love)
- 1985 C++ (Bjarne Stroustrup)

aktuell

- 2024 C23 & C++23
- weitere Entwicklung in Arbeitsgruppen
  - ISO/IEC/JTC 1/SC 22/WG 14 für C
  - ISO/IEC/JTC 1/SC 22/WG 21 für C++



Interessenskonflikte & Spannungsfelder

- Compiler
  - maximale Performance
  - schnelle Übersetzungszeit
  - wenig Interesse an Austauschbarkeit
- Hardware
  - geringer Aufwand
  - hohe Kompatibilität mit verschiedenen Compilern
- Software
  - Rückwärtskompatibilität
  - Lesbarkeit
  - Wiederverwendbarkeit
  - Interoperabilität
  - Fehlervermeidung

Motivation für C und C++

- Grundlage vieler modernen Programmiersprachen
  - Syntax Basis für z.B. C#, D, Java, Javascript
- hardwarenahe mit guten Abstraktionskonzepten
- sehr performant
  - z.B. für embedded, Echtzeitsysteme, Game-Engines
- viele existierende Systeme in C
  - Betriebssystemkerne Linux, Windows, MacOS
  - Datenbanken Oracle, PostgreSQL, SQLite, Redis
- Compiler für (fast)
  - jedes System und
  - jede Architektur
- Grundlegendes Verständnis für Mechanismen hinter modernen Abstraktionen
  - z.B. Verwendung von Bibliotheken
- Interoperabilität (Brücke) zwischen Sprachen
  - Foreign Function Interface (FFI)
  - z.B. Java Foreign Function and Memory API
    - früher Java Native Interface Specification: Contents

Ablauf



8+8 Einheiten C

- Grundlagen
- Übersetzungsvorgang
- Zeiger und dynamischer Speicher
- Funktionen
- Bibliotheken
- Interoperabilität
- Fokus auf low-level



20+20 Einheiten C++

- Grundlagen
- objektorientierte Programmierung
- idiomatisches C++
- effiziente und sichere Ressourcenverwaltung
- Generizität und Schablonen
- Smart-Pointer
- Ausnahmen
- Container und Algorithmen
- Modernes C++
- Fokus auf high-level

Beurteilung

Vorlesung

- schriftliche Klausur am Semesterende (90min)
- ohne Unterlagen
- auf Papier

Übung

- 4 Hausübungen
  - Fokus auf Engagement
- 2 Kurztests
  - am Computer aber ohne IDE
- Mitarbeit
  - Fragen, Kommentare, ...